# Digital Queue Management and Table-Aware Seating in Restaurant Operations: A Simulation-Based Evaluation of a Restaurant Queue System

**Anjun Saini[1], Abhinav Kumar[2], Abhinav Choudhary[3], Harsh Dhonkariya[4], Ishika Sharma[5], Er. Ram Babu Buri[6], Dr. Vishal Shrivastava[7], Dr. Akhil Pandey[8]**

[1,2,3,4,5]*Department of Artificial Intelligence & Data Science,* [6,7,8]*Department of Computer Science & Engineering, Arya College of Engineering & I.T., Jaipur, India*

*anjunsaini.24ai@aryacollege.in, abhinavkumar.24ai@aryacollege.in, abhinavchoudhary.24ai@aryacollege.in, harshdhonkriya.24ai@aryacollege.in, ishikasharma.24ai@aryacollege.in, rambabuburi.cs@aryacollege.in, vishalshrivastava.cs@aryacollege.in, akhil@aryacollege.in*

## Abstract

Restaurant queues are still largely managed with paper lists, verbal wait-time quotes and called names. These practices produce unreliable estimates, avoidable walk-aways, idle tables caused by no-shows and seating decisions that ignore the match between party size and table capacity. This paper proposes a digital Restaurant Queue System (RQS) that combines remote waitlisting through QR and web channels, a projection-based waiting-time estimator, a table-aware seating policy with a bounded-bypass fairness guarantee, and an automated notification workflow that shortens table holds and reduces no-shows. The system is decomposed into loosely coupled services and accompanied by a privacy-by-design and Zero-Trust-style security baseline aligned with India's Digital Personal Data Protection Act, 2023. Because a field deployment was outside the scope of this study, the design is evaluated through a discrete-event simulation of a 20-table, 74-seat restaurant, whose engine is verified against the Erlang-C formula, using 1,000 paired replications for each of four demand scenarios. Compared with a manual first-fit baseline, the RQS seated 8.7% to 11.1% more guests, reduced no-show losses by about 65%, and lowered the error of waiting-time quotes by 28% to 58%. A component ablation identifies the notification workflow as the largest contributor, and a sensitivity analysis shows that a 7.1% throughput gain remains even when guests are assumed to be no more patient than in a manual queue. The paper also reports a trade-off: under heavy load, parties that are seated wait slightly longer, because fewer of them abandon.

**Keywords:** RESTAURANT QUEUE MANAGEMENT, WAITLIST, QUEUEING THEORY, DISCRETE-EVENT SIMULATION, WAIT-TIME ESTIMATION, TABLE ASSIGNMENT, NO-SHOW REDUCTION, SERVICE OPERATIONS, DIGITAL HOSPITALITY.

## 1. Introduction

Restaurants are capacity-constrained service systems. A fixed number of tables, each with a fixed number of seats, must serve a stream of guests whose arrivals are bursty, whose party sizes vary, and whose patience is limited. When demand exceeds capacity, which happens every weekend evening in most urban casual-dining venues, a queue forms at the door. How that queue is managed decides how many guests are eventually seated, how long they wait, how fairly they are treated, and how much revenue the venue earns from its most perishable asset: table time.

In many establishments the queue is still handled by a paper list, a verbal quote such as "about twenty minutes," and a host who calls names across a crowded entrance. This arrangement is cheap but fragile. Wait-time promises depend on the host's intuition, guests must stay physically close to the venue so as not to miss their call, and the host cannot easily see which waiting party best fits the table that has just been vacated. The outcome is familiar to every diner: walk-aways, hovering crowds, tables left idle while a called party is searched for, and large tables occupied by small parties.

Digital waitlisting offers an alternative. A guest scans a QR code or opens a web page, registers a party size and a phone number, and receives a live estimate and status updates. Behind the interface, the venue obtains a structured record of every arrival, quote, call and seating, which supports both operational decisions and analytics. However, a digital front-end alone does not guarantee better outcomes. Its value depends on the quality of the wait-time estimates it produces, the seating policy it applies, the way it handles no-shows, and the privacy and security of the guest data it collects.

This paper therefore treats the restaurant queue as an engineering and operations-research problem rather than as a user-interface problem. We design a Restaurant Queue System (RQS) built from loosely coupled services, formalize its behavior with queueing-theoretic models, and evaluate it against a manual baseline using a discrete-event simulation whose engine we verify against closed-form results. All performance results reported here are simulation outputs under stated assumptions, and we are explicit about which assumptions drive which conclusions.

### 1.1 Motivation & Problem Statement

Three operational failure modes characterize manually managed restaurant queues:

1. **Unreliable Waiting-Time Information:** Quotes given at

the door are based on rules of thumb that ignore party-size mismatch and the progress of parties already dining. Inaccurate quotes lead guests either to leave unnecessarily or to wait far longer than promised, and they erode trust in the venue.

2. **Capacity-Blind Seating:** A host usually seats the first party that fits whichever table has just become free. This is fair in the sense of arrival order but ignores efficiency: a six-seat table may be given to a party of two while a party of six waits, which wastes seat capacity and lengthens the queue for large groups.
3. **No-Shows and Idle Table Time:** When a party is called and does not respond, the table sits unused during the search. Guests who must remain within earshot of the host are also more likely to give up, since waiting in a crowded doorway is an unoccupied and uncertain experience.

### 1.2 Research Questions

To address these bottlenecks, this research investigates three research questions:

**RQ1:** How can a digital queue system produce more accurate waiting-time quotes than a host's rule-of-thumb estimate, and how much does estimate accuracy matter for lost demand?

**RQ2:** Can a table-aware seating policy with a bounded-bypass fairness guarantee increase the number of guests seated without allowing any party to be starved?

**RQ3:** What are the quantifiable operational benefits of the complete RQS (estimation, seating and notification) compared with a manual first-fit queue under weekday, weekend, surge and group-heavy demand, and which components contribute most?

### 1.3 Major Contributions

The key engineering and scientific contributions of this paper are:

1. A modular RQS architecture comprising a customer channel, queue service, estimator and seating engine, table-state manager, notification service, host dashboard and analytics, with an explicit queue-party state machine.
2. A projection-based waiting-time estimator and a table-aware best-fit seating policy with a bounded-bypass rule (parameter $K$) that guarantees no party is overtaken more than $K$ times by later arrivals for a table it could have used.
3. A verified discrete-event simulation framework, validated against the Erlang-C formula, with common random numbers, 1,000 paired replications per scenario, a load sweep, a component ablation and a sensitivity analysis on the most influential behavioral assumption.
4. A privacy-by-design and Zero-Trust-style security baseline for guest data, mapped to India's Digital Personal Data Protection Act, 2023, and a STRIDE threat model of the queue platform.

## 2. Literature Review

Research relevant to restaurant queue management spans queueing theory, the behavioral science of waiting, hospitality revenue management and, more recently, digital service platforms. This section synthesizes these strands and identifies the gap that the present work addresses.

### 2.1 Queueing Theory Foundations

Queueing analysis of service systems begins with Erlang's work on telephone exchanges [2], which produced the delay formula that still bears his name and models a multi-server system with Poisson arrivals and exponential service. Kleinrock [3] and Gross et al. [4] provide the standard treatments of the M/M/$c$ family. Little's law [1] links the mean number in a system, the arrival rate and the mean time in the system without distributional assumptions, and it remains the most widely used consistency check in queue studies.

Plain Erlang-C assumes that customers wait indefinitely. In practice, restaurant guests balk when the quote seems too long and renege when the wait exceeds their tolerance. Garnett, Mandelbaum and Reiman [11] showed that explicitly modeling abandonment (the Erlang-A model) changes both performance predictions and staffing decisions, and Zeltyn and Mandelbaum [12] analyzed many-server asymptotics with general patience distributions. Gans, Koole and Mandelbaum [13] surveyed these ideas for call centers; the same structure of balking and reneging appears in restaurants, but with a defining difference: servers (tables) are heterogeneous in capacity, so customers are not interchangeable.

### 2.2 Psychology of Waiting and Delay Information

Maister [5] proposed that perceived waits are lengthened when time is unoccupied, when the wait is uncertain or unexplained, and when it feels unfair. Larson [6] argued that first-in-first-out order is valued because it signals social justice, and that violations of it provoke disproportionate dissatisfaction. Taylor [7] found that delays affect service evaluations through uncertainty and anger, and Davis and Heineke [8] showed that the disconfirmation between expected and actual waits explains satisfaction better than the actual wait alone. Hui and Tse [24] and Whitt [23] studied what and how much delay information to give customers, and Ibrahim and Whitt [25] developed real-time delay estimators based on delay history. Together these works motivate the two design principles of the RQS: give guests an accurate and timely estimate, and keep departures from arrival order small and bounded.

### 2.3 Restaurant Revenue Management and Table Mix

Kimes et al. [9] introduced revenue-management concepts to restaurants, notably revenue per available seat hour (RevPASH) and the management of meal duration. Kimes and Thompson [10] used a table-mix analysis at a restaurant chain to show that the combination of table sizes materially affects the number of guests that can be served. These works establish that matching party size to table capacity is economically important, but they study the layout and duration side of the

**Table 1:** Taxonomy and Comparison of Restaurant Queue Management Paradigms

| Parameter / Feature | Paper List + Verbal Call | Pager / Buzzer | Proposed Digital RQS |
|---|---|---|---|
| Wait information | Host intuition | Host intuition | Projection-based estimate, live updates |
| Guest must stay on premises | Yes | Nearby range only | No (remote waiting) |
| No-show handling | Repeated calling | Re-page | Pre-alert + timed hold + auto-release |
| Table-fit awareness | Manual judgment | Manual judgment | Automated best-fit with bounded bypass |
| Operational data captured | None | Minimal | Full event log and analytics |
| Fairness safeguard | Informal | Informal | Explicit $K$-bypass rule |
| Set-up cost | Very low | Hardware cost | Software + SMS cost |

**Table 2:** Literature Review Matrix on Waiting, Queueing and Restaurant Operations

| Study & Year | Focus Area | Key Finding | Limitation |
|---|---|---|---|
| Little (1961) [1] | Queue identity | $L = \lambda W$ holds for general stable systems | Averages only |
| Maister (1985) [5] | Psychology of waiting | Unoccupied, uncertain and unfair waits feel longer | Conceptual |
| Larson (1987) [6] | Queue fairness | FIFO order carries social-justice value | Not a seating model |
| Taylor (1994) [7] | Delay and service evaluation | Uncertainty and anger shape evaluations of delay | Non-restaurant setting |
| Davis & Heineke (1998) [8] | Perceived vs. actual wait | Disconfirmation drives satisfaction | Survey-based |
| Garnett et al. (2002) [11] | Impatient customers | Modeling abandonment changes staffing results | Call-center context |
| Kimes & Thompson (2004) [10] | Table mix | Table-size mix affects guests served | Static layout focus |
| Ibrahim & Whitt (2009) [25] | Delay estimation | Real-time estimators from delay history | Single-class queue |

problem more than the dynamic assignment of waiting parties to freed tables. Our seating policy targets this latter decision.

### 2.4 Digital Waitlists and Notification Channels

Commercial waitlist applications are widely used, yet there is little published, reproducible evidence on how their algorithmic components (estimation, seating and reminders) individually affect outcomes. The choice of notification channel also matters, because it determines whether guests receive the call in time. Table 3 compares common channels qualitatively.

The literature therefore leaves a gap: estimation, seating and notification have each been studied in isolation or in other industries, but their combined effect in a heterogeneous-table restaurant, and the contribution of each component, has not been quantified in an open, reproducible framework. This paper addresses that gap.

**Table 3:** Qualitative Comparison of Notification Channels for Table-Ready Alerts

| Channel | Guest Needs App | Reach | Delivery Confirmation | Recurring Cost |
|---|---|---|---|---|
| SMS | No | Very high | Delivery receipt | Per message |
| WhatsApp Business | WhatsApp | High in India | Read receipt | Per conversation |
| Push notification | Yes (PWA / app) | Medium | Limited | Low |
| Voice call | No | High | Answered / not | Per minute |
| Buzzer / pager | No (hardware) | Range-limited | Physical | Capital + loss |

**Table 4:** Notation Used in the Theoretical Model

| Symbol | Meaning |
|---|---|
| $\lambda,\ \mu,\ c$ | Arrival rate, service rate per table, number of tables |
| $a,\ \rho$ | Offered load $a = \lambda/\mu$ and utilization $\rho = a/c$ |
| $s_p,\ c_t$ | Size of party $p$; capacity of table $t$ |
| $\theta_p$ | Patience of party $p$ (Gamma-distributed) |
| $q_p,\ W_p$ | Quoted wait and realized wait of party $p$ |
| $b_p,\ K$ | Bypass count of party $p$; bypass limit |
| $\gamma$ | Aging weight in the seating score (per minute) |
| $g$ | Patience multiplier for remote waiting |

## 3. Theoretical Framework

This section formalizes the queueing behavior of the RQS: the baseline multi-server model used for verification, the abandonment mechanism, the waiting-time estimator and the seating rule. Table 4 summarizes the notation.

### 3.1 Multi-Server Queue and Erlang-C Verification Model

If all parties had the same size and tables were identical, the restaurant would reduce to an M/M/$c$ queue with Poisson arrivals at rate $\lambda$, exponential dining times with rate $\mu$ and $c$ tables. The offered load is $a = \lambda/\mu$ and the utilization is $\rho = a/c$, with stability requiring $\rho < 1$. The probability that an arriving party must wait is given by the Erlang-C formula and the mean waiting time follows from it:

$$C(c,a) = \frac{\dfrac{a^c}{c!}\cdot\dfrac{1}{1-\rho}}{\displaystyle\sum_{k=0}^{c-1}\frac{a^k}{k!} + \frac{a^c}{c!}\cdot\frac{1}{1-\rho}}, \tag{1}$$

$$W_q = \frac{C(c,a)}{c\mu - \lambda}. \tag{2}$$

We use this closed form only as an independent check on the simulation engine (Section 5.4). The real restaurant departs from M/M/$c$ in three ways: party sizes and table capacities are heterogeneous, parties abandon, and dining durations depend on party size.

### 3.2 Balking, Reneging and Remote Waiting

Each party $p$ has a patience $\theta_p$ drawn from a Gamma distribution with shape 4, which is more realistic than the exponential

because very few guests give up within a minute or two. On arrival the party receives a quote $q_p$. If $q_p > \theta_p$ the party balks and never joins; otherwise it joins and reneges if its realized wait reaches $\theta_p$. Because a digital queue lets guests wait elsewhere, we scale patience by a multiplier $g \geq 1$ for digital waiting ($g = 1.25$ as the base case). Since $g$ is a behavioral assumption rather than a measured value, Section 6.6 varies it from 1.0 to 1.5.

### 3.3 Little's Law as a Consistency Check

$$L_q = \lambda_{\text{eff}} \cdot W_q, \tag{3}$$

where $\lambda_{\text{eff}}$ is the rate of parties that actually join the queue. The relation links the observed mean queue length to the mean wait of joining parties. It also shows why a lower mean wait of served parties can coexist with more lost demand: the mean is taken over those who stay.

### 3.4 Projection-Based Waiting-Time Estimator

For a new party of size $s$, the estimator computes a virtual schedule. Each occupied table $t$ is assigned an expected release time

$$f_t = \max\big(\sigma_t + \mathbb{E}[D \mid s_t] + \tau,\ \text{now} + \delta\big), \tag{4}$$

where $\sigma_t$ is the seating time, $\mathbb{E}[D \mid s] = 30 + 6s$ minutes is the expected dining duration, $\tau$ is the cleaning time and $\delta$ a short grace term. Tables on hold release at the end of the hold. Waiting parties ahead are then placed, in order, on the fitting table with the earliest release time, advancing that table's release by their expected occupancy. The quote is

$$q_{\text{new}} = \min_{t\,:\,c_t \geq s} f_t - \text{now}. \tag{5}$$

The baseline host estimate is the common rule of thumb

$$q_{\text{host}} = \frac{(n_{\text{ahead}} + 1) \cdot D_{\text{avg}}}{c}, \tag{6}$$

rounded to 5 minutes, where $D_{\text{avg}}$ is the mean dining time across party sizes. It captures queue length but not party-size mismatch.

### 3.5 Table-Aware Seating with Bounded Bypass

When table $t$ becomes available, the baseline host seats the first waiting party that fits. The RQS instead scores each eligible party $p$ (those with $s_p \leq c_t$):

$$\text{Score}(p, t) = \frac{s_p}{c_t} + \gamma \cdot (\text{now} - a_p). \tag{7}$$

The first term rewards seat utilization and the second ages the party by its waiting time ($\gamma = 0.015$ per minute). To keep the policy fair, each party carries a bypass counter $b_p$ that is incremented whenever a later arrival is seated at a table that $p$ could have used. Once $b_p \geq K$ ($K = 3$), $p$ takes priority over all others for any table that fits it. By construction, no party is overtaken more than $K$ times by later arrivals for tables it could have used, which is the formal fairness guarantee of the policy. Free tables are examined from smallest to largest so that large tables are kept for large parties when possible.

### 3.6 Cost of a No-Show

If a called party is absent with probability $\pi$, the table is held for $H$ minutes and then released, so the expected idle table time per call is $\pi H$, and every seating also incurs a response delay $\delta_r$ before the party is actually seated. With the manual values ($\pi = 0.12$, $H = 4$, $\delta_r = 2.0$) versus the RQS values ($\pi = 0.04$, $H = 2$, $\delta_r = 0.5$), the expected lost table time per call, $\pi H + \delta_r$, falls from 2.48 to 0.58 minutes. The saving looks small per call, but it recurs at every table turn, and a table turns many times in an evening.

## 4. Proposed System Architecture

The proposed RQS decomposes the queueing workflow into specialized services behind a single gateway, so that each concern (state, estimation, notification, analytics) can be developed, scaled and secured independently.

### 4.1 Customer Channel and Onboarding

A QR code at the entrance and on the menu opens a lightweight web page, implemented as a progressive web application so that no installation is required [16]. The guest enters a name, a party size, an optional seating preference and a mobile number, which is verified with a one-time password to deter fake entries. The guest then sees the position in line, the projected wait, and a Cancel option, and the page updates in real time over a WebSocket connection [15]. Walk-in guests without a phone can be added by staff through the host dashboard.

### 4.2 Decoupled Service Modules

The application layer is decomposed into six functional modules:

1. **Queue Service:** Creates, updates and cancels queue entries and enforces the party state machine of Table 7.
2. **Estimator & Seating Engine:** Computes quotes (Section 3.4) and chooses the party to seat when a table frees (Section 3.5).
3. **Table-State Manager:** Tracks each table as available, occupied, in cleaning or on hold, and exposes release events to the engine.
4. **Notification Service:** Sends the join confirmation, the "ten minutes away" pre-alert, the "table ready" call and reminders through the channels in Table 3.
5. **Host Dashboard:** A tablet view showing the live queue, table map and one-tap actions (seat, mark no-show, add walk-in, adjust a table).
6. **Analytics & Reporting:** Aggregates waits, no-shows, abandonment and table utilization for managers.

### 4.3 Queue Entry State Machine

Every queue entry follows an explicit lifecycle that makes the system auditable and prevents inconsistent states (for example, a seated party that is also marked as cancelled). Table 7 lists the states and transitions. The pre-alert (NOTIFIED) is sent when the estimate falls below ten minutes, giving the guest time to return; the hold timer starts at the table-ready call.

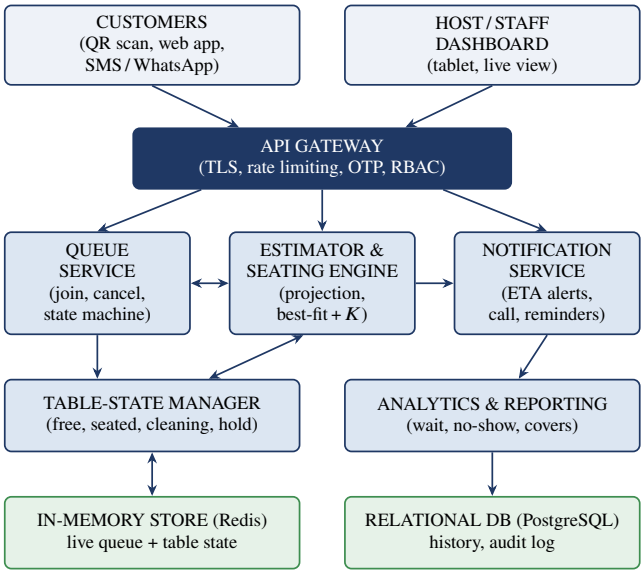



*Figure 1: Architectural Overview of the Proposed Restaurant Queue System (RQS).*

**Table 5:** Reference Technology Stack (Adaptable to the Project Implementation)

| Layer | Technology | Rationale |
|---|---|---|
| Client | React PWA, QR entry | No install; works on any phone |
| API | REST [22] + WebSocket [15] | Simple CRUD plus live updates |
| Live state | Redis (sorted sets) | Ordered queue, fast reads |
| Persistence | PostgreSQL | Durable history and audit trail |
| Messaging | SMS / WhatsApp gateway | Reaches guests without an app |
| Security | TLS, OTP, JWT, RBAC | Guest verification and staff roles |
| Deployment | Docker containers | Reproducible, portable releases |

**Table 6:** Service Responsibilities and Design Targets (Targets Are Design Goals, Not Measurements)

| Module | Primary Input | Output | Design Target |
|---|---|---|---|
| Queue Service | Join / cancel requests | Queue entry, state | Ordered, consistent state |
| Estimator | Table and queue state | Quote per party | Recomputed on every change |
| Seating Engine | Table-free event | Party–table match | Decision in milliseconds |
| Notification | State transitions | SMS / WhatsApp | Delivery receipts logged |
| Dashboard | Staff actions | Live table map | One-tap operations |
| Analytics | Event log | KPIs, reports | Daily and weekly views |

### 4.4 Seating Engine Procedure

The seating engine runs whenever a table is released, a party joins or a party leaves. Its logic, which is the exact policy simulated in Section 5, is:

1. Collect the set of available tables and the list of waiting parties in arrival order.

**Table 7:** Queue-Entry States, Transitions and Timers

| State | Entered When | Exits To | Timer |
|---|---|---|---|
| WAITING | Guest joins | NOTIFIED, CANCELLED | None |
| NOTIFIED | ETA $\leq$ 10 min | CALLED, CANCELLED | None |
| CALLED | Table ready | SEATED, NO_SHOW | Hold: 2 min |
| SEATED | Host confirms | COMPLETED | Dining + cleaning |
| NO_SHOW | Hold expires | Terminal | Table released |
| CANCELLED | Guest leaves | Terminal | Slot released |

2. Examine available tables from smallest to largest; for each, form the eligible set $E$ of waiting parties with $s_p \leq c_t$. Skip the table if $E$ is empty.
3. If any party in $E$ has $b_p \geq K$, select the earliest-arrived such party. Otherwise select the party with the highest score from Section 3.5.
4. Increment $b_q$ for every party $q$ in $E$ that arrived earlier than the selected party.
5. Send the "table ready" call, start the hold timer, and mark the table as on hold. On host confirmation the party becomes SEATED; on timer expiry it becomes NO_SHOW and the table is released, which triggers step 1 again.

### 4.5 Data Model and Event Log

The persistent model has four main entities: PARTY (identifier, size, hashed contact, preference, join channel), QUEUE_ENTRY (party reference, state, timestamps, quote, bypass count), TABLE (identifier, capacity, zone, current state) and EVENT (append-only record of every join, quote, notification, call, seating, cancellation and no-show). The event log is the single source for analytics, for auditing disputed seating order, and for calibrating dining-time and patience distributions in a later pilot.

## 5. Experimental Setup & Simulation

A field deployment requires a partner restaurant, staff training and weeks of data collection, none of which was available to this study. We therefore evaluate the RQS with a discrete-event simulation that follows standard simulation methodology [14, 21]. The same arrival streams, party sizes, patience draws and dining-time draws are used for both policies (common random numbers), so that every reported difference is a paired comparison.

### 5.1 Simulated Restaurant and Parameters

The simulated venue has 20 tables and 74 seats (eight two-seat, eight four-seat, three six-seat and one eight-seat table). Arrivals follow a non-homogeneous Poisson process over a four-hour dinner window whose intensity profile in successive hours is 0.5, 0.9, 1.0 and 0.7 of the peak rate. Table 8 lists the parameters. The parameter values are assumptions chosen to represent a mid-sized casual-dining restaurant and are not measurements from a specific venue.

**Table 8:** Simulation Parameters for the Manual Baseline and the Proposed RQS

| Parameter | Manual Baseline | Proposed RQS |
|---|---|---|
| Tables / seats | 20 / 74 | 20 / 74 |
| Dining time (min) | $30 + 6s$, lognormal ($\sigma = 0.35$) | Same |
| Cleaning time | 5 min | 5 min |
| Patience (mean) | Gamma(4), 30 min | Gamma(4), 37.5 min ($g = 1.25$) |
| Wait quote | Host rule of thumb | Projection-based |
| Seating rule | First-fit in arrival order | Best-fit score, $K = 3$ |
| No-show probability | 0.12 | 0.04 |
| Hold before release | 4 min | 2 min |
| Response delay | 2.0 min | 0.5 min |
| Replications | 1,000 (paired) | 1,000 (paired) |

**Table 9:** Demand Scenarios

| Scenario | Peak Rate | Party-Size Mix | Special Condition |
|---|---|---|---|
| A: Weekday | 24 parties/h | Standard | Moderate load |
| B: Weekend peak | 40 parties/h | Standard | Sustained overload |
| C: Surge | 40 parties/h | Standard | +40 parties in 20 min |
| D: Group-heavy | 30 parties/h | More parties of 5+ | Large-table pressure |

### 5.2 Demand Scenarios

Four scenarios exercise the system under different stress conditions (Table 9). Scenario B is the principal case because it creates sustained overload, which is when queue management matters most.

The standard mix assigns 5%, 33%, 15%, 25%, 8%, 7%, 4% and 3% to parties of size 1 to 8; the group-heavy mix assigns 3%, 20%, 12%, 25%, 15%, 12%, 7% and 6%.

### 5.3 Policies Compared

The manual baseline uses the host quote of Section 3.4, a first-fit seating rule, a 12% chance that a called party is absent, a four-minute hold and a two-minute response delay. The RQS uses the projection-based quote, the best-fit seating rule with $K = 3$, a 4% chance of absence thanks to the pre-alert, a two-minute hold and a half-minute response delay. The RQS also assumes that guests tolerate longer waits ($g = 1.25$) because they are free to wait elsewhere. This last assumption is the one with the least empirical support, so it is isolated in the ablation and sensitivity studies.

### 5.4 Verification of the Simulation Engine

To verify the engine, we reconfigured it as a homogeneous M/M/20 queue (identical tables, unit party size, exponential dining with mean 50 minutes, no cleaning, no abandonment) and compared the simulated mean wait with the Erlang-C value for three utilization levels, using five long runs each (Table 10). All simulated means agree with the closed-form values within the 95% intervals.

**Table 10:** Verification of the Simulation Engine Against the Erlang-C Formula ($c = 20$)

| Utilization $\rho$ | Erlang-C $W_q$ (min) | Simulated $W_q$ (min) | 95% Half-Width |
|---|---|---|---|
| 0.6 | 0.15 | 0.15 | ± 0.03 |
| 0.8 | 3.20 | 3.05 | ± 1.08 |
| 0.9 | 13.77 | 14.15 | ± 3.73 |

**Table 11:** Scenario B (Weekend Peak): Mean Results over 1,000 Paired Replications

| Performance Metric | Manual | Proposed RQS | Difference |
|---|---|---|---|
| Guests seated (covers) | 247.2 | 270.7 | +23.5 |
| Table utilization (%) | 67.4 | 74.5 | +7.1 |
| Mean wait, seated (min) | 9.1 | 10.2 | +1.2 |
| 90th-pct. wait (min) | 23.5 | 26.1 | +2.6 |
| Balked (% arrivals) | 17.5 | 21.9 | +4.4 |
| Reneged (% arrivals) | 10.5 | 3.8 | −6.6 |
| Total abandonment (%) | 28.0 | 25.8 | −2.2 |
| No-shows (% arrivals) | 8.6 | 3.0 | −5.7 |
| Quote error (min) | 7.4 | 5.1 | −2.3 |
| Wait, parties of 5+ (min) | 17.7 | 15.2 | −2.6 |
| Max bypass (avg. of maxima) | 0 | 2.97 | Bounded by $K = 3$ |

### 5.5 Performance Measures and Statistical Design

For each replication we record: guests seated (covers), mean and 90th-percentile wait of seated parties, abandonment (balking plus reneging) and no-show rates as shares of arrivals, table utilization (dining time within the service window divided by table time available), the mean absolute error between quoted and realized wait, and the maximum bypass count. We report means with 95% confidence half-widths across replications, and paired differences where appropriate.

## 6. Results and Discussion

The simulation results indicate that the RQS improves throughput, reduces no-show losses and gives guests more reliable estimates, at the cost of a modest increase in the mean wait of served parties under heavy load.

### 6.1 Weekend-Peak Performance

Table 11 shows that in Scenario B the RQS seated 23.5 more guests (a 9.5% increase), raised table utilization from 67.4% to 74.5%, and reduced reneging from 10.5% to 3.8% of arrivals. No-shows fell from 8.6% to 3.0%. The mean wait of seated parties rose from 9.1 to 10.2 minutes, an effect we discuss in Section 6.4. Figure 2 shows that the throughput advantage appears at every demand level, grows with load, and is accompanied by lower abandonment at moderate and high demand levels (at the lightest load the two policies are indistinguishable).

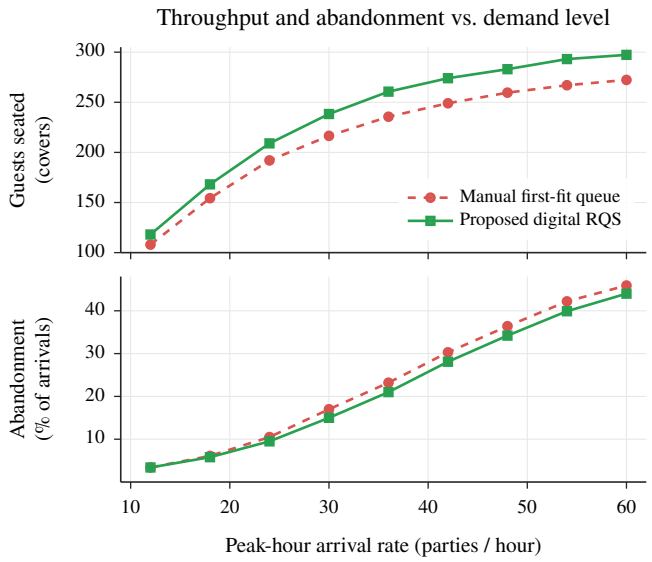



*Figure 2: Guests Seated and Abandonment Across Peak-Hour Arrival Rates (400 Replications per Point).*

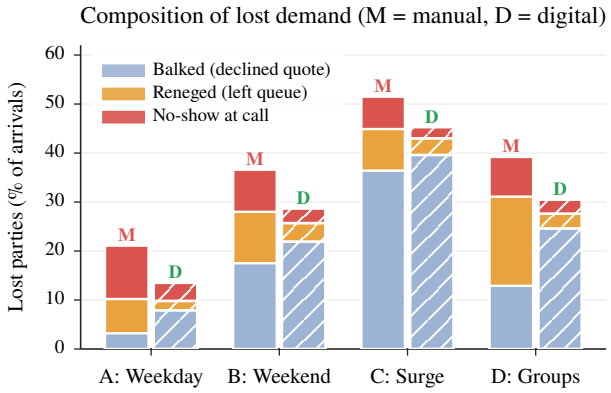



*Figure 3: Composition of Lost Demand by Scenario (Balk, Renege, No-Show).*

### 6.2 Where Lost Demand Goes: Balking, Reneging and No-Shows

Figure 3 decomposes lost demand. Total abandonment falls modestly in every scenario (for example from 28.0% to 25.8% in Scenario B), but its composition changes substantially. Reneging, in which a guest waits and then gives up, falls sharply under the RQS, whereas balking rises. The reason is informational: an accurate quote allows guests who would not have been willing to wait to decline at the door instead of occupying the queue and leaving later. For the venue this is a better failure mode, since a declined quote costs the guest no time, and the queue is not padded with parties that will not stay. No-show losses fall by about 65% on average across scenarios.

### 6.3 Accuracy of Waiting-Time Quotes (RQ1)

The projection-based estimator reduces the mean absolute quote error by 28% to 58% across the four scenarios (Figure 4);

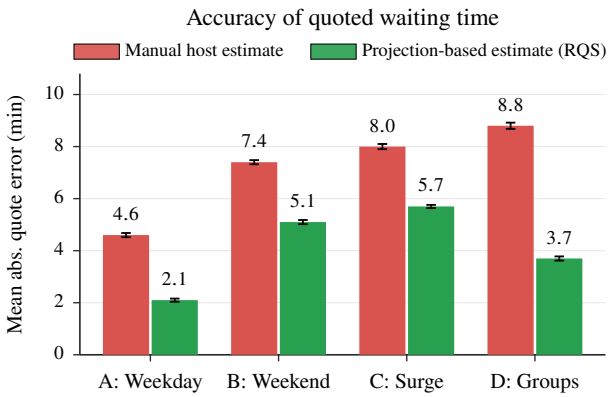



*Figure 4: Mean Absolute Error of Quoted Waiting Time (Error Bars: 95% Confidence).*

the improvement is largest in the group-heavy scenario (58%), where party-size mismatch makes the host's rule of thumb least reliable. The estimator is not perfect, because it cannot anticipate future abandonments, no-shows or the variability of dining time, but its error is consistently lower. The ablation in Section 6.5 shows that better quotes mainly act on the composition of lost demand rather than on throughput.

### 6.4 Waiting Time and Fairness (RQ2)

Under light load (Scenario A) the RQS reduces the mean wait of served parties from 4.3 to 3.5 minutes. Under heavy load (Scenarios B and C) the mean wait of seated parties is higher with the RQS, by 1.2 and 3.0 minutes respectively. This is a selection effect rather than a slowdown: the RQS retains guests who would have reneged under the manual process, and these are exactly the parties with long waits. Figure 5 shows that the RQS distribution is shifted to the right in its tail for this reason. Operators who prefer shorter waits to higher throughput can cap the maximum wait the system will quote.

Fairness is bounded by design. The manual first-fit rule never lets a later arrival take a table that an earlier party could have used, so its bypass count is zero by construction. The RQS permits such overtakes for efficiency but caps them: the largest bypass count observed in any replication of Scenarios B and C was 3 and 3, never above $K = 3$. Large parties (five or more) wait less with the RQS in all four scenarios; in Scenario B their mean wait falls from 17.7 to 15.2 minutes, because the score rule steers large tables to large parties.

### 6.5 Component Ablation (RQ3)

To attribute the gain to its sources, each RQS component was added alone to the manual baseline in Scenario B (Table 12, Figure 6). The notification workflow, which lowers the no-show probability, hold time and response delay, is the largest single contributor (+5.4%). The patience assumption contributes +2.9%, table-aware seating +1.6%, and the projection-based quote contributes essentially nothing to throughput (+0.0%) but lowers total abandonment from 28.2% to 25.0%. The sum of the individual effects (9.8%) is close to the full-system gain of 9.5%, suggesting that the components act almost independently.

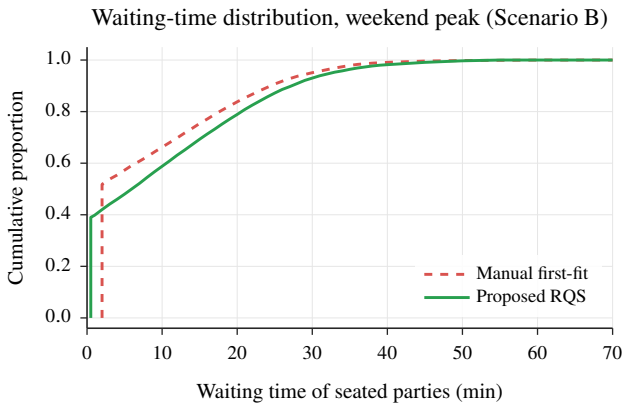



*Figure 5: Empirical Distribution of Waiting Times of Seated Parties in Scenario B.*

**Table 12:** Component Ablation in Scenario B (Mean over 1,000 Paired Replications)

| Configuration | Covers | Gain (%) | Aband. (%) | No-show (%) |
|---|---|---|---|---|
| Manual baseline | 247.4 | — | 28.2 | 8.4 |
| + Wait-time projection | 247.4 | +0.0 | 25.0 | 8.8 |
| + Table-aware seating | 251.4 | +1.6 | 28.1 | 8.5 |
| + Notification workflow | 260.7 | +5.4 | 30.1 | 2.8 |
| + Patience ($g$ = 1.25) | 254.5 | +2.9 | 26.5 | 8.7 |
| Full RQS | 270.8 | +9.5 | 25.9 | 3.0 |

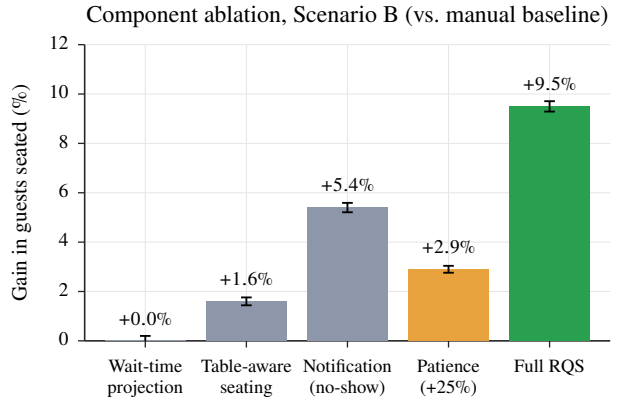



*Figure 6: Contribution of Each RQS Component to the Increase in Guests Seated (Scenario B).*

### 6.6 Sensitivity to the Patience Assumption

The multiplier $g$ is the least certain parameter. Figure 7 varies it from 1.0 (no extra patience) to 1.5. Even at $g = 1.0$ the RQS seats 7.1% more guests than the manual baseline, so the principal result does not depend on this assumption, although its size does: the gain rises to 12.5% at $g = 1.5$. Mean wait of seated parties rises with $g$ for the reason explained in Section 6.4.

### 6.7 Results Across All Scenarios

The gain in guests seated ranges from 8.7% to 11.1% across scenarios (Table 13), and every paired difference in covers and in total abandonment is statistically distinguishable from

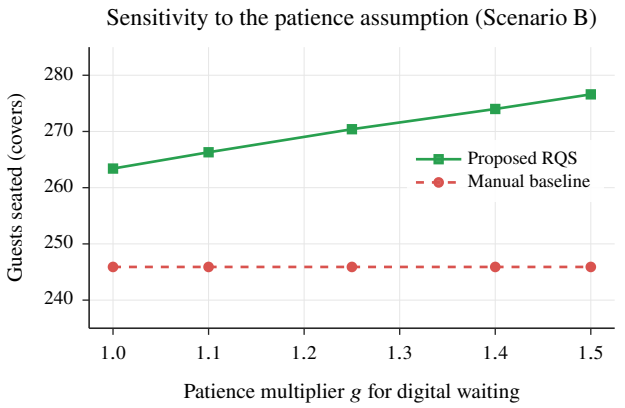



*Figure 7: Sensitivity of Guests Seated to the Patience Multiplier g (Scenario B).*

**Table 13:** Summary Across the Four Demand Scenarios (Manual / RQS)

| Scenario | Covers | Gain | No-show % | Quote Err. (min) |
|---|---|---|---|---|
| A: Weekday | 192 / 208 | +8.7% | 10.9 / 3.7 | 4.6 / 2.1 |
| B: Weekend | 247 / 271 | +9.5% | 8.6 / 3.0 | 7.4 / 5.1 |
| C: Surge | 250 / 273 | +9.3% | 6.6 / 2.3 | 8.0 / 5.7 |
| D: Groups | 206 / 229 | +11.1% | 8.1 / 2.9 | 8.8 / 3.7 |

zero at the 95% level (the paired half-widths are below 0.7 covers and 0.3 percentage points). The advantage is largest when party sizes are diverse and tables are scarce (Scenario D), which is where table-aware seating and accurate quotes matter most.

### 6.8 Threats to Validity and Limitations

First, all results derive from simulation with assumed parameters; absolute values will differ in a real venue, and the figures should be read as relative comparisons between policies under identical conditions. Second, the guest-behavior model (Gamma patience, balk-on-quote) is a simplification, and the patience multiplier and no-show probabilities of the RQS are assumptions that should be calibrated through a pilot. Third, the estimator assumes dining durations depend only on party size, ignoring course timing and bill requests. Fourth, reservations, remote joining before arrival, table combining and takeaway demand are excluded. Fifth, the model does not capture the staff workload of operating the dashboard. These limitations motivate the future work in Section 8.

### 6.9 Practical Deployment Guidance

The results suggest a staged rollout. First, deploy the notification workflow and host dashboard, because the ablation identifies no-show reduction as the largest and least assumption-dependent gain. Second, enable the projection-based quote, which improves the honesty of the queue and shifts lost demand from reneging to balking. Third, enable table-aware seating once staff are comfortable, starting with $K = 2$ or 3 and monitoring the fairness complaints reported by hosts. Throughout, the venue should track five indicators from the

**Table 14:** Security Controls, Threat Mitigation and Regulatory Alignment

| Control Domain | Implementation | Threat Mitigated | Alignment |
|---|---|---|---|
| Consent and minimization | Notice, minimal fields, timestamp | Over-collection | DPDP Act 2023 |
| In-transit protection | TLS everywhere | Eavesdropping | OWASP Top 10 |
| Guest verification | OTP + rate limiting | Fake-queue flooding | OWASP Top 10 |
| Access control | RBAC, strong staff auth | Privilege misuse | OWASP Top 10 |
| Retention | Delete / hash after visit | Data exposure | DPDP, GDPR |
| Audit logging | Append-only event log | Repudiation | Good practice |

event log: seated covers per service, table utilization, no-show rate, reneging rate and mean absolute quote error. A two-to-four-week pilot at a partner restaurant would calibrate the behavioral parameters assumed in this study.

## 7. Security, Privacy & Compliance

A waitlist collects personal data (names and phone numbers) from members of the public, so security and privacy are design requirements rather than add-ons.

### 7.1 Data Minimization, Consent and Retention

The RQS follows the principle of collecting only what the service needs: a first name or nickname, a party size and a verified mobile number. A short notice at joining states the purpose (queue communication), and consent is recorded with a timestamp. Under India's Digital Personal Data Protection Act, 2023 [18] the venue acts as a data fiduciary, and the design supports purpose limitation, erasure on request and retention limits. Phone numbers are deleted or irreversibly hashed after the visit, after which only aggregate analytics are kept. Where guests from the European Union may be served, the same controls support the principles of the GDPR [19].

### 7.2 Transport Security, Access Control and Abuse Resistance

All client and staff traffic uses TLS. Staff accounts use role-based access control (host, manager, administrator) so that only managers can export reports. Joining requires a one-time password, and the gateway applies per-number and per-device rate limits to prevent fake entries that would inflate the queue. QR codes are printed on tamper-evident material because a replaced code could redirect guests to a malicious page. Dependencies and container images are scanned in the build pipeline, and the design is checked against the OWASP Top 10 [20].

### 7.3 Threat Modeling via STRIDE

The STRIDE framework [17] was applied to the queue platform. Table 15 lists representative threats, the controls in the design and a qualitative residual-risk rating assigned by the authors during design review (not a measured value).

**Table 15:** STRIDE Threat Modeling Matrix for the Queue Platform

| STRIDE | Threat Vector | Mitigation Control | Residual Risk |
|---|---|---|---|
| Spoofing | Fake guest or staff identity | OTP, staff authentication | Low |
| Tampering | Altering queue order | Server-side rules, RBAC, audit log | Low |
| Repudiation | Disputed seating order | Append-only event log | Low |
| Info Disclosure | Leak of phone numbers | TLS, minimization, deletion | Medium–Low |
| Denial of Service | Flood of fake joins | Rate limits, OTP | Medium–Low |
| Elevation of Privilege | Host account gaining admin rights | RBAC, least privilege | Low |

## 8. Conclusion

This paper presented a Restaurant Queue System that combines remote digital waitlisting, a projection-based waiting-time estimator, a table-aware seating policy with a bounded-bypass fairness guarantee, and a notification workflow that shortens table holds and reduces no-shows. The design is modular and is accompanied by a privacy and security baseline suited to the handling of guest contact data.

Because the evaluation relies on a verified discrete-event simulation rather than a field trial, the results should be interpreted as relative comparisons under stated assumptions. Within those limits, the RQS seated 8.7% to 11.1% more guests than a manual first-fit queue across four demand scenarios, reduced no-show losses by about 65%, and lowered the error of waiting-time quotes by 28% to 58%. The ablation shows that the notification workflow is the largest contributor, that table-aware seating adds a smaller but consistent gain, and that the accuracy of quotes mainly changes how demand is lost (declining at the door rather than abandoning later). Sensitivity analysis shows that a gain of 7.1% remains even when guests are assumed to be no more patient than in a manual queue. The system has a cost: under heavy load, the mean wait of served parties is longer because fewer parties give up.

Future work includes a pilot deployment at a partner restaurant to calibrate patience, no-show and dining-time parameters; extending the model to reservations, remote joining before arrival and table combining; learning dining-time distributions from the event log to improve the estimator; and a human-factors study of host dashboard workload and guest satisfaction.

## References


[1] Little, J. D. C. (1961). A proof for the queuing formula: $L = \lambda W$. *Operations Research*, *9*(3), 383–387.

[2] Erlang, A. K. (1917). Solution of some problems in the theory of probabilities of significance in automatic telephone exchanges. *The Post Office Electrical Engineers' Journal*, *10*, 189–197.

[3] Kleinrock, L. (1975). *Queueing Systems, Volume 1: Theory*. Wiley.

[4] Gross, D., Shortle, J. F., Thompson, J. M., & Harris, C. M. (2008). *Fundamentals of Queueing Theory* (4th ed.). Wiley.

[5] Maister, D. H. (1985). The psychology of waiting lines. In J. A. Czepiel, M. R. Solomon, & C. F. Surprenant (Eds.), *The Service Encounter* (pp. 113–123). Lexington Books.

[6] Larson, R. C. (1987). Perspectives on queues: Social justice and the psychology of queueing. *Operations Research*, *35*(6), 895–905.

[7] Taylor, S. (1994). Waiting for service: The relationship between delays and evaluations of service. *Journal of Marketing*, *58*(2), 56–69.

[8] Davis, M. M., & Heineke, J. (1998). How disconfirmation, perception and actual waiting times impact customer satisfaction. *International Journal of Service Industry Management*, *9*(1), 64–73.

[9] Kimes, S. E., Chase, R. B., Choi, S., Lee, P. Y., & Ngonzi, E. N. (1998). Restaurant revenue management: Applying yield management to the restaurant industry. *Cornell Hotel and Restaurant Administration Quarterly*, *39*(3), 32–39.

[10] Kimes, S. E., & Thompson, G. M. (2004). Restaurant revenue management at Chevys: Determining the best table mix. *Decision Sciences*, *35*(3), 371–392.

[11] Garnett, O., Mandelbaum, A., & Reiman, M. (2002). Designing a call center with impatient customers. *Manufacturing & Service Operations Management*, *4*(3), 208–227.

[12] Zeltyn, S., & Mandelbaum, A. (2005). Call centers with impatient customers: Many-server asymptotics of the M/M/*n*+G queue. *Queueing Systems*, *51*(3–4), 361–402.

[13] Gans, N., Koole, G., & Mandelbaum, A. (2003). Telephone call centers: Tutorial, review, and research prospects. *Manufacturing & Service Operations Management*, *5*(2), 79–141.

[14] Banks, J., Carson, J. S., Nelson, B. L., & Nicol, D. M. (2010). *Discrete-Event System Simulation* (5th ed.). Prentice Hall.

[15] Fette, I., & Melnikov, A. (2011). *The WebSocket Protocol* (RFC 6455). Internet Engineering Task Force.

[16] ISO/IEC. (2015). *Information technology — Automatic identification and data capture techniques — QR Code bar code symbology specification* (ISO/IEC 18004:2015).

[17] Shostack, A. (2014). *Threat Modeling: Designing for Security*. Wiley.

[18] Government of India. (2023). *The Digital Personal Data Protection Act, 2023* (Act No. 22 of 2023).

[19] European Union. (2016). *Regulation (EU) 2016/679 (General Data Protection Regulation)*.

[20] OWASP Foundation. (2021). *OWASP Top 10:2021 — The Ten Most Critical Web Application Security Risks*.

[21] Law, A. M. (2015). *Simulation Modeling and Analysis* (5th ed.). McGraw-Hill Education.

[22] Fielding, R. T. (2000). *Architectural Styles and the Design of Network-based Software Architectures* (Doctoral dissertation). University of California, Irvine.

[23] Whitt, W. (1999). Improving service by informing customers about anticipated delays. *Management Science*, *45*(2), 192–207.

[24] Hui, M. K., & Tse, D. K. (1996). What to tell consumers in waits of different lengths: An integrative model of service evaluation. *Journal of Marketing*, *60*(2), 81–90.

[25] Ibrahim, R., & Whitt, W. (2009). Real-time delay estimation based on delay history. *Manufacturing & Service Operations Management*, *11*(3), 397–415.